

SIH PS 34 — Legal Metrology Automation

System Integration Status & Architectural Mapping Report

Executive Overview: This report details the current integration status of the system. The core foundation — consisting of **Member 1 (NestJS Backend)**, **Member 5 (Legal Knowledge Base)**, and **Member 3 (Flutter Mobile App)** — has been fully connected and verified. The remaining roles (Members 2, 4, and 6) are architecturally mapped to plug in seamlessly without breaking existing mobile app flows.

1. COMPLETED INTEGRATIONS — WHO IS CONNECTED & HOW?

A. Member 5 (Legal Knowledge Base) → Member 1 (NestJS Backend)

Status: 100% CONNECTED IN CODE & SEEDED

- **Legal Rulebook & Penalty Seeder:** Member 1's seeder script (backend/src/database/seeder/seed-legal-kb.ts) directly loads and parses Member 5's rulebook.json (53 KB), penalty_matrix.json, compounding_matrix.json, and exemptions.json.
- **Bilingual Notice Generator:** Member 1's notice generator service (backend/src/modules/notices/notice-generator.service.ts) directly loads Member 5's markdown legal templates (notice_improvement_en.md, notice_seizure_mr.md, etc.) and dynamically substitutes all 24 legal placeholders (e.g. {{NOTICE_ID}}, {{BUSINESS_NAME}}, {{OBSERVED_VIOLATION}}, {{DEADLINE}}).

B. Member 1 (NestJS Backend) → Member 3 (Flutter Mobile App)

Status: 100% CONNECTED VIA REST REPOSITORIES & DUAL MODE

- **Single Base URL Gateway:** Flutter ApiClient (lib/core/network/api_client.dart) attaches Keycloak JWT Bearer <token> headers to every HTTP request. Isolated token refresh prevents infinite 401 interceptor loops.
- **Live Repository Mappers:** Flutter RealXxxRepository classes (lib/data/real/real_repositories.dart) target Member 1's NestJS routes (/auth/login, /businesses, /inspections, /notices, /payments, /self-check, /offences).
- **Dual Mode Architecture:** Runs in **Demo Mode** (USE MOCK DATA=true) on-device standalone, or flips to **Live Mode** (USE MOCK DATA=false) with zero UI code changes.

C. Member 3 (Mobile App) State Sync & Rule Alignment

Status: VERIFIED & TESTED (8/8 TESTS PASSED)

- **Violation Propagation:** Connected ViolationsStep state to InspectionFlowScreen. Confirmed violations now properly flow into NoticeStep for AI notice generation.
- **Flutter 3.32/3.33 Deprecations:** Replaced deprecated DropdownButtonFormField.value with initialValue and cleaned up static analysis warnings across all screens.

2. REMAINING TASKS & FUTURE INTEGRATION WIRING PLAN

A. Member 2: AI/OCR + NLP Engineer (FastAPI Service)

Remaining Deliverable: FastAPI endpoints for /ocr/scan, /ocr/self-check, and /notices/generate.

Integration Protocol: Member 1's NestJS backend calls FastAPI internally. The Mobile App calls NestJS — **zero mobile app changes required** when Member 2 deploys!

B. Member 4: Web Frontend Developer (Next.js — Citizen + Controller)

Remaining Deliverable: Citizen complaint filing form & Controller regional analytics/compounding approval web app.

Integration Protocol: Next.js app communicates with Member 1's NestJS backend. When Controller assigns inspections or approves notices on Web, updates sync in PostgreSQL and reflect in the Flutter Mobile App.

C. Member 6: Integrations, QA & Presentation Owner (Razorpay & End-to-End QA)

Remaining Deliverable: Razorpay sandbox keys, /payments/webhook verification, end-to-end QA, and pitch deck.

Integration Protocol: Mobile App payment screen initiates Razorpay order via Member 1; Webhook updates payment status in PostgreSQL.

3. SYSTEM INTEGRATION MATRIX SUMMARY

Member / Role	Status	Connected To	Integration Method
Member 1 Backend & DB Architect	COMPLETED	Member 5 (KB) Member 3 (Mobile)	Prisma PostgreSQL, Keycloak JWT, File Seeder, REST Controllers
Member 5 Legal KB Maker	COMPLETED	Member 1 (Backend)	Structured JSON Rulebook, CSV Penalty Matrix, Bilingual Markdown Templates

Member 3 Mobile App (Flutter)	COMPLETED	Member 1 (Backend)	Riverpod, Dio REST Repositories, Dual Mock/Real Switch, Signature Pad
Member 2 AI/OCR + NLP Engineer	IN PROGRESS	Member 1 (Backend)	FastAPI internal proxy called by NestJS (Zero Mobile UI changes)
Member 4 Web Frontend (Next.js)	IN PROGRESS	Member 1 (Backend)	REST endpoints for Citizen complaints & Controller compounding approvals
Member 6 Integrations & QA	IN PROGRESS	Member 1 (Backend) Member 3 (Mobile)	Razorpay checkout SDK & NestJS /payments/webhook